

CLINE_RULES_USER_GUIDE

Cline Rules System - Comprehensive User Guide

HelixCode Phase 2, Feature 2

Version: 1.0 Last Updated: November 7, 2025 Status: Complete

Table of Contents

1. [Introduction](#)
 2. [Quick Start](#)
 3. [Rule File Format](#)
 4. [Pattern Matching](#)
 5. [Rule Inheritance](#)
 6. [Rule Categories](#)
 7. [Rule Metadata](#)
 8. [Advanced Features](#)
 9. [Best Practices](#)
 10. [Examples](#)
 11. [API Reference](#)
 12. [Troubleshooting](#)
 13. [FAQ](#)
-

Introduction

The Cline Rules System provides a flexible, hierarchical way to define project-specific guidelines, conventions, and requirements that LLMs should follow when generating or modifying code.

What are Cline Rules?

Cline Rules are project-level instructions stored in `.clinerules` files that tell HelixCode how to handle different files in your project. They can specify:

- **Code style conventions** (formatting, naming, structure)
- **Architecture guidelines** (patterns, dependencies, organization)

- **Security requirements** (authentication, validation, encryption)
- **Testing standards** (coverage, frameworks, patterns)
- **Documentation requirements** (comments, docstrings, READMEs)
- **Performance guidelines** (optimization, caching, algorithms)

Key Features

- **3-Level Hierarchy:** Workspace → Project → File-specific rules
- **Pattern Matching:** Glob, regex, exact, and wildcard patterns
- **Priority-Based:** Rules are scored and applied by priority
- **Flexible Format:** Simple YAML-like syntax
- **Auto-Detection:** Pattern types detected automatically
- **Thread-Safe:** Concurrent access support
- **Extensible:** Custom metadata and categories

Why Use Cline Rules?

1. **Consistency:** Ensure all generated code follows project conventions
 2. **Quality:** Enforce best practices and standards automatically
 3. **Documentation:** Centralize project guidelines in one place
 4. **Flexibility:** Different rules for different parts of codebase
 5. **Inheritance:** Share common rules, override when needed
 6. **Automation:** LLMs automatically follow project standards
-

Quick Start

Creating Your First Rule File

Create a `.clinerules` file in your project root:

```
# Project-level rules for MyProject

[Go Style]
pattern: *.go
description: Go code style guidelines
category: style
priority: 5
```

Follow these Go conventions:

- Use gofmt for formatting
- Error handling: always check errors, never ignore
- Naming: use camelCase for unexported, PascalCase for exported
- Comments: every exported function must have a doc comment

```
[Test Requirements]
pattern: *_test.go
description: Testing standards
category: testing
priority: 10
```

All test files must:

- Use table-driven tests for multiple cases
- Include both positive and negative test cases
- Achieve at least 80% code coverage
- Use testify for assertions

Using Rules Programmatically

```
package main

import (
    "dev.helix.code/internal/rules"
)

func main() {
    // Create a rule manager
    manager := rules.NewManager()

    // Load rules from project directory
    err := manager.LoadFromDirectory(".")
    if err != nil {
        panic(err)
    }

    // Get rules for a specific file
    matches := manager.GetRulesForFile("internal/server/
        handler.go")

    // Format rules as a prompt for LLM
    prompt := manager.FormatRulesForFile("internal/server/
        handler.go")
    fmt.Println(prompt)
}
```

Directory Structure

```
myproject/
├── .helix/
```

```
|   └─ .clinerules           # Workspace-level rules (highest scope)
├─ .clinerules              # Project-level rules
├─ src/
│   └─ api/
│       └─ api.clinerules    # File-specific rules for API module
│       └─ utils/
│           └─ utils.clinerules # File-specific rules for utilities
└─ README.md
```

Rule File Format

Basic Structure

A `.clinerules` file consists of rules, each with a name, optional metadata, and content:

```
[Rule Name]
key: value
another_key: value
```

Rule content goes here.
Multiple lines are supported.

Rule Components

Each rule has:

1. **Name** (required): Enclosed in `[brackets]`
2. **Metadata** (optional): Key-value pairs
3. **Content** (required): The actual guideline text

Supported Metadata Keys

Key	Type	Description	Example
pattern	string	File matching pattern	<code>*.go, src/**/*.js</code>
description	string	Rule description	Code style for Go
priority	integer	Rule priority (1-100)	10
category	string	Rule category	style, security
scope	string	Application scope	

Key	Type	Description	Example
			global, directory, file
tags	comma-separated	Tags for filtering	backend, critical

Comments

Lines starting with # are comments:

```
# This is a comment
[Rule Name]
# This is also a comment
pattern: *.go
```

Complete Example

```
# Backend API Guidelines
```

```
[API Structure]
pattern: src/api/**/*.go
description: REST API endpoint structure
priority: 10
category: architecture
scope: directory
tags: api, backend, http
```

All API endpoints must:

1. Use the Gin framework
2. Implement proper error handling with HTTP status codes
3. Validate all input parameters
4. Return JSON responses in standard format:

```
{
  "success": true|false,
  "data": {...},
  "error": "error message if applicable"
}
```

```
[Authentication]
pattern: src/api/*/handler.go
description: Authentication requirements
priority: 20
category: security
tags: security, auth, critical
```

Every handler must:

- Check authentication using the auth middleware
 - Verify user permissions for the operation
 - Log all access attempts with user ID
 - Use JWT tokens for authentication
-

Pattern Matching

Pattern Types

HelixCode supports four pattern types, which are **auto-detected** from the pattern syntax:

1. Glob Pattern (Default)

Syntax: Contains * or ? wildcards

Examples:

```
pattern: *.go           # Matches all Go files
pattern: *_test.go      # Matches all test files
pattern: config.*.yaml  # Matches config.dev.yaml, config.prod.yaml
pattern: src/**/*.js    # Matches all JS files under src/ at any depth
```

Glob Wildcards: - * - Matches any characters except / - ** - Matches zero or more directory levels - ? - Matches exactly one character

Examples:

Pattern	Matches	Doesn't Match
*.go	main.go, test.go	main.js, dir/file.go
src/**/*.go	src/main.go, src/api/handler.go	test/main.go
test_?.go	test_1.go, test_a.go	test_10.go

2. Regex Pattern

Syntax: Enclosed in /slashes/

Examples:

```
pattern: /\.*\.go$/           # All files ending in .go
pattern: /^src\/.*_handler\.go$/ # Handlers in src directory
pattern: /test_[0-9]+\.go$/     # Numbered test files
```

Use Cases: - Complex pattern matching - Character classes and ranges - Lookaheads and lookbehinds - Capturing groups (for metadata)

3. Exact Pattern

Syntax: No wildcards, no slashes

Examples:

```
pattern: src/main.go          # Only matches this exact file
pattern: config/production.yaml # Only this specific config
```

Use Cases: - File-specific rules - Critical files requiring special handling - Override rules for specific files

4. Any Pattern

Syntax: * or ** alone

Examples:

```
pattern: *                    # Matches all files
pattern: **                   # Matches all files (same as *)
```

Use Cases: - Default rules - Workspace-level guidelines - Fallback patterns

Pattern Matching Examples

```
[Backend Go Files]
pattern: src/**/*.go
# Matches:
#   src/main.go
#   src/api/handler.go
#   src/api/middleware/auth.go
```

```
[Test Files Only]
pattern: *_test.go
# Matches:
#   handler_test.go
#   utils_test.go
# Doesn't match:
#   api/handler_test.go (not in root)
```

```
[Specific Config]
pattern: config/production.yaml
# Matches only:
#   config/production.yaml
```

```
[All TypeScript React]
pattern: /\.*\.(tsx|ts)$/
# Matches:
#   App.tsx
#   utils.ts
#   components/Button.tsx
```

Pattern Priority

When multiple patterns match a file, rules are scored and sorted by:

1. **Pattern specificity** (exact > glob > regex > any)
 2. **Rule priority** (explicit priority value)
 3. **Scope** (file > directory > global)
 4. **Inheritance level** (file-specific > project > workspace)
-

Rule Inheritance

Three-Level Hierarchy

Rules are inherited and merged across three levels:

Workspace (.helix/.clinerules)	# Lowest priority, broadest scope
↓	
Project (.clinerules)	# Medium priority
↓	
File-specific (*.clinerules)	# Highest priority, most specific

How Inheritance Works

1. **All matching rules are collected** from all three levels
2. **Rules are scored** based on:
 - Pattern specificity
 - Explicit priority
 - Scope
 - Inheritance level (file +3000, project +2000, workspace +1000)
3. **Rules are sorted** by score (highest first)
4. **LLM receives all matching rules** in priority order

Inheritance Example

Workspace Rule (.helix/.clinerules):

```
[General Style]
pattern: *
priority: 1
```

Use consistent formatting and naming conventions.

Project Rule (.clinerules):

```
[Go Style]
pattern: *.go
priority: 5
```

Follow Go conventions:

- Use gofmt
- Document exported symbols

File Rule (api/api.clinerules):

```
[API Handlers]
pattern: *_handler.go
priority: 10
```

API handlers must:

- Validate input
- Return standard JSON format

For file `api/user_handler.go`, all three rules apply, in this order: 1. API Handlers (file-specific, priority 10, score ~3010) 2. Go Style (project-level, priority 5, score ~2005) 3. General Style (workspace-level, priority 1, score ~1001)

Overriding Rules

File-specific rules can override project or workspace rules by:

1. **Using same rule name** (latest wins)
2. **Higher priority value** (explicit override)
3. **More specific pattern** (automatic priority)

Example override:

Project (.clinerules):

```
[Error Handling]
pattern: *.go
priority: 5
```

Always return errors explicitly.

File (legacy/legacy.clinerules):

```
[Error Handling]
pattern: *.go
priority: 10
```

Legacy code: panic() is acceptable for critical errors.

For files in legacy/, the file-specific rule takes precedence.

Rule Categories

Built-in Categories

HelixCode provides standard categories for organizing rules:

Category	Purpose	Examples
style	Code formatting and conventions	Indentation, naming, structure
architecture	System design and patterns	MVC, layering, dependencies
security	Security requirements	Auth, validation, encryption
testing	Testing standards	Coverage, frameworks, patterns
documentation	Documentation requirements	Comments, READMEs, API docs
performance	Performance guidelines	Caching, algorithms, optimization
general	General guidelines	Default category

Using Categories

In rule files:

```
[Auth Security]
category: security
pattern: src/auth/**/*.go
```

All authentication code must:

- Use bcrypt for password hashing
- Implement rate limiting
- Log all authentication attempts

In code:

```
// Get all security rules
securityRules :=
    manager.GetRulesByCategory(rules.RuleCategorySecurity)

// Check if critical security rules apply
for _, rule := range securityRules {
    if hasTag(rule, "critical") {
        // Enforce strictly
    }
}
```

Custom Categories

You can define custom categories:

```
[Database Schema]
category: database
pattern: migrations/*.sql
```

All migrations must:

- Be reversible (include DOWN migration)
- Include transaction wrappers
- Update version tracking

Rule Metadata

Priority

Priority determines rule importance (1-100, default: 1).

```
[Critical Security]
priority: 100
pattern: src/auth/**/*.go
```

```
[Code Style]
priority: 5
pattern: *.go
```

```
[General Guidelines]
priority: 1
pattern: *
```

Higher priority rules appear first when multiple rules match.

Scope

Scope defines where a rule applies:

Scope	Description	Example Use Case
global	Entire project	General code style
directory	Specific directory tree	Module-specific conventions
file	Individual files	Special file requirements

```
[Project Standards]
scope: global
pattern: *
```

```
[API Module Standards]
scope: directory
pattern: src/api/**/*
```

```
[Main Entry Point]
scope: file
pattern: src/main.go
```

Tags

Tags enable filtering and grouping:

```
[Authentication]
tags: security, backend, critical
pattern: src/auth/**/*go
```

```
[Frontend Style]
tags: frontend, react, style
pattern: src/components/**/*tsx
```

Querying by tags:

```
// Get all critical rules
criticalRules := manager.GetRulesByTag("critical")

// Get all backend security rules
backendSecurity := filterByTags(
    manager.GetRulesByCategory(rules.RuleCategorySecurity),
    []string{"backend", "critical"},
)
```

Custom Metadata

Add any custom key-value pairs:

```
[API Versioning]
pattern: src/api/v*/**/*.go
api_version: v2
deprecated: false
owner: backend-team
review_required: true
```

API endpoints must follow v2 standards.

Accessing custom metadata:

```
for _, match := range matches {
    if match.Rule.Metadata["review_required"] == "true" {
        // Flag for manual review
    }
}
```

Advanced Features

Multi-File Rules

A single `.clinerules` file can contain rules for multiple file patterns:

```
[Go Files]
pattern: *.go
Use Go conventions.
```

```
[Test Files]
pattern: *_test.go
```

Use table-driven tests.

[Config Files]

pattern: *.yaml

Use YAML best practices.

Dynamic Rule Loading

Load rules at runtime:

```
manager := rules.NewManager()
```

```
// Load from directory (finds all .clinerules)
```

```
manager.LoadFromDirectory("./myproject")
```

```
// Load specific rule files
```

```
manager.LoadWorkspaceRules(".helix/.clinerules")
```

```
manager.LoadProjectRules(".clinerules")
```

```
manager.LoadFileRules("api", "api/api.clinerules")
```

```
// Add rules programmatically
```

```
rule := &rules.Rule{
```

```
    ID:          "dynamic-rule-1",
```

```
    Name:        "Dynamic Rule",
```

```
    Pattern:     "*.go",
```

```
    PatternType: rules.PatternTypeGlob,
```

```
    Content:     "Dynamic content",
```

```
    Priority:    10,
```

```
}
```

```
manager.AddProjectRule(rule)
```

Rule Export and Import

Export all rules to a single file:

```
// Export
```

```
combined := manager.Export()
```

```
content := rules.Format(combined)
```

```
os.WriteFile("all-rules.clinerules", []byte(content), 0644)
```

```
// Import
```

```
parser := rules.NewParser("all-rules.clinerules")
```

```
ruleSet, _ := parser.Parse()
```

Rule Validation

Validate rules before applying:

```
rule := &rules.Rule{
    Name:      "Test Rule",
    Pattern:    "[invalid(regex", // Invalid regex
    PatternType: rules.PatternTypeRegex,
    Content:    "Content",
}

if err := rule.Validate(); err != nil {
    fmt.Printf("Invalid rule: %v\n", err)
    // Output: Invalid rule: invalid regex pattern: ...
}
```

Thread-Safe Operations

All manager operations are thread-safe:

```
manager := rules.NewManager()

// Safe concurrent access
go func() {
    manager.AddWorkspaceRule(rule1)
}()

go func() {
    matches := manager.GetRulesForFile("main.go")
}()

go func() {
    count := manager.Count()
}()
```

Best Practices

1. Organize by Hierarchy

Use the three-level hierarchy effectively:

- **Workspace:** Organization-wide standards
- **Project:** Project-specific conventions
- **File:** Module or component-specific rules

2. Use Appropriate Priorities

Reserve priority ranges for different needs:

- **1-20:** General guidelines
- **21-50:** Important conventions
- **51-80:** Critical requirements
- **81-100:** Security and safety rules

x

3. Be Specific

Write clear, actionable rules:

Bad:

☐

```
[Code Quality]
pattern: *
Write good code.
```

Good:

```
[Error Handling]
pattern: src/**/*.go
priority: 40
category: architecture
```

Error handling requirements:

1. Always check errors returned from functions
2. Wrap errors with context using `fmt.Errorf("context: %w", err)`
3. Return errors up the call stack
4. Only handle errors at boundaries (HTTP handlers, main)
5. Never use `panic()` except for truly unrecoverable errors

4. Use Categories and Tags

Organize rules for easy filtering:

```
[SQL Injection Prevention]
category: security
tags: database, security, critical
pattern: src/**/*.go
```

Prevent SQL injection:

- Use parameterized queries
- Never concatenate user input into SQL
- Use prepared statements

5. Version Your Rules

Track rule changes in version control:

```
# .clinerules
# Version: 2.1
# Last updated: 2025-11-07
# Changes: Added security requirements for auth module

[Authentication]
# Added in version 2.1
pattern: src/auth/**/*.*go
...
```

6. Test Your Patterns

Verify patterns match expected files:

```
rule := &rules.Rule{
    Pattern:      "src/**/*.*go",
    PatternType:  rules.PatternTypeGlob,
}

testCases := []struct {
    path      string
    matches   bool
}{
    {"src/main.go", true},
    {"src/api/handler.go", true},
    {"test/main.go", false},
}

for _, tc := range testCases {
    if rule.Matches(tc.path) != tc.matches {
        log.Printf("Pattern failed for %s", tc.path)
    }
}
```

7. Document Rule Intent

Explain why rules exist:

```
[No Direct Database Access in Handlers]
pattern: src/api/*/handler.go
priority: 60
```

category: architecture

DO NOT access the database directly from HTTP handlers.

Rationale:

- Separates concerns (handler = HTTP, service = business logic)
- Makes code testable (can mock service layer)
- Enables transaction management at service level
- Allows reuse of business logic from other entry points

Instead, inject a service and call its methods:

```
service.GetUser(id)
db.Query("SELECT * FROM users WHERE id = ?", id)
```

8. Regular Review

Schedule regular rule reviews:

- Quarterly: Review all rules for relevance
- Monthly: Update rules based on new learnings
- Weekly: Add rules for newly discovered patterns
- On incidents: Create rules to prevent recurrence

Examples

Example 1: Microservices Project

Microservices Architecture Rules

```
[Service Structure]
pattern: services/*/
priority: 50
category: architecture
tags: microservices, structure
```

Each service must be self-contained:

- Own database schema
- Independent deployment
- API versioning (v1, v2, etc.)
- Health check endpoint at /health
- Metrics endpoint at /metrics

```
[Service Communication]
```

pattern: services/*/client/*.go
priority: 60
category: architecture
tags: microservices, communication

Inter-service communication:

- Use HTTP/JSON for synchronous calls
- Use message queue (RabbitMQ) for async
- Implement circuit breaker pattern
- Add request tracing with OpenTelemetry
- Timeout all external calls (default: 30s)

[Service Tests]

pattern: services/*/**/*_test.go
priority: 40
category: testing
tags: microservices, testing

Integration tests must:

- Use testcontainers for dependencies
- Test against real database
- Mock external service calls
- Test failure scenarios
- Achieve 80% coverage minimum

Example 2: React Frontend

React Frontend Guidelines

[Component Structure]

pattern: src/components/**/*_tsx
priority: 30
category: architecture
tags: frontend, react, components

React component structure:

1. Use functional components with hooks
2. One component per file
3. Export component as default
4. Props interface defined above component
5. Styled-components for styling

Example:

```
interface ButtonProps {
```

```
    onClick: () => void;
    label: string;
    variant?: 'primary' | 'secondary';
  }
```

```
export default function Button({ onClick, label, variant = 'primary' }: BU
  return <StyledButton variant={variant} onClick={onClick}>{label}</Styled
}
```

```
[State Management]
pattern: src/**/*.tsx
priority: 40
category: architecture
tags: frontend, react, state
```

State management rules:

- Use useState for local component state
- Use useContext for shared state (theme, user, etc.)
- Use React Query for server state
- Never prop drill more than 2 levels
- Avoid Redux unless absolutely necessary

```
[Component Testing]
pattern: src/**/*.test.tsx
priority: 35
category: testing
tags: frontend, react, testing
```

Component tests must:

- Use React Testing Library
- Test user interactions, not implementation
- Mock API calls with MSW
- Test loading and error states
- Achieve 90% coverage for components

Example 3: Security-Critical Application

Security-Critical Application Rules

```
[Input Validation]
pattern: src/**/*.go
priority: 90
category: security
tags: security, validation, critical
```

ALL user input must be validated:

1. Type validation (string, int, email, etc.)
2. Length limits (prevent buffer overflow)
3. Format validation (regex for email, phone, etc.)
4. Whitelist allowed characters
5. Sanitize HTML content

Use validator library:

```
import "github.com/go-playground/validator/v10"
```

[Authentication]

pattern: src/auth/**/*.go

priority: 100

category: security

tags: security, auth, critical

Authentication requirements:

- Use bcrypt with cost factor 12 for passwords
- Implement rate limiting (5 attempts per 15 minutes)
- Use JWT with RS256 (not HS256)
- Rotate JWT secrets every 90 days
- Log all authentication attempts
- Implement 2FA for admin accounts

[Data Protection]

pattern: src/**/*.go

priority: 95

category: security

tags: security, data, critical, gdpr

Data protection (GDPR compliance):

- Encrypt all PII at rest (AES-256)
- Use TLS 1.3 for data in transit
- Never log sensitive data (passwords, tokens, PII)
- Implement right to deletion (data purging)
- Implement data export functionality
- Audit all access to sensitive data

[Dependency Security]

pattern: go.mod

priority: 85

category: security

tags: security, dependencies

Dependency management:

- Run `go mod tidy` before commits
- Use `govulncheck` in CI pipeline
- Update dependencies monthly
- No dependencies with known CVEs
- Pin versions (no floating versions)

Example 4: Legacy Code Migration

Legacy Code Migration Rules

[Legacy Code]

pattern: legacy/**/*.go

priority: 20

tags: legacy, technical-debt

Legacy code guidelines:

- DO NOT refactor unless fixing a bug
- Add TODO comments for future improvements
- New features go in modern codebase
- Plan for eventual deprecation

[Modern Code]

pattern: src/**/*.go

priority: 40

tags: modern, best-practices

Modern codebase standards:

- Use interfaces for dependencies
- Dependency injection via constructors
- Unit tests for all new code
- Follow SOLID principles
- Use context for cancellation

[Migration Path]

pattern: src/adapter/**/*.go

priority: 50

category: architecture

tags: migration, adapter

Adapters between legacy and modern:

- Implement adapter pattern
- Translate legacy structs to modern types

- Add integration tests for adapters
 - Document translation logic
 - Plan for adapter removal
-

API Reference

Core Types

```
// Rule represents a project rule
type Rule struct {
    ID            string           // Unique identifier
    Name          string           // Human-readable name
    Description    string           // Rule description
    Pattern       string           // File pattern
    PatternType   PatternType      // Pattern type (glob/regex/
                                exact/any)
    Content       string           // Rule content/guidelines
    Priority       int             // Priority (higher = more
                                important)
    Category      RuleCategory    // Rule category
    Scope         RuleScope      // Application scope
    Tags          []string        // Tags for filtering
    Metadata      map[string]string // Custom metadata
}

// Manager manages rules with inheritance
type Manager struct {
    // Private fields
}

// RuleSet represents a collection of rules
type RuleSet struct {
    Name          string
    Description    string
    Rules         []*Rule
    Metadata      map[string]string
}

// MatchResult represents a rule match
type MatchResult struct {
    Matches bool
    Score   int
    Rule    *Rule
}
```

```
    FilePath string
}
```

Pattern Types

```
const (
    PatternTypeGlob  PatternType = "glob"    // Glob pattern (*.go)
    PatternTypeRegex PatternType = "regex"    // Regular expression
    PatternTypeExact PatternType = "exact"    // Exact file path
    PatternTypeAny   PatternType = "any"      // Matches any file
)
```

Categories

```
const (
    RuleCategoryStyle      RuleCategory = "style"
    RuleCategoryArchitecture RuleCategory = "architecture"
    RuleCategorySecurity   RuleCategory = "security"
    RuleCategoryTesting     RuleCategory = "testing"
    RuleCategoryDocumentation RuleCategory = "documentation"
    RuleCategoryPerformance RuleCategory = "performance"
    RuleCategoryGeneral     RuleCategory = "general"
)
```

Scopes

```
const (
    RuleScopeGlobal    RuleScope = "global"    // Entire project
    RuleScopeDirectory RuleScope = "directory" // Specific
    directory
    RuleScopeFile      RuleScope = "file"      // Specific file
)
```

Manager Methods

```
// Create a new manager
func NewManager() *Manager

// Load rules from directory (finds all .clinerules files)
func (m *Manager) LoadFromDirectory(dir string) error

// Load workspace rules
func (m *Manager) LoadWorkspaceRules(path string) error

// Load project rules
```



```
func (m *Manager) LoadProjectRules(path string) error

// Load file-specific rules
func (m *Manager) LoadFileRules(fileKey, path string) error

// Get all rules matching a file (with inheritance)
func (m *Manager) GetRulesForFile(filePath string) []*MatchResult

// Get all rules from all levels
func (m *Manager) GetAllRules() []*Rule

// Get rules by category
func (m *Manager) GetRulesByCategory(category RuleCategory)
    []*Rule

// Get rules by tag
func (m *Manager) GetRulesByTag(tag string) []*Rule

// Format rules as LLM prompt
func (m *Manager) FormatRulesForFile(filePath string) string

// Add rule to workspace level
func (m *Manager) AddWorkspaceRule(rule *Rule) error

// Add rule to project level
func (m *Manager) AddProjectRule(rule *Rule) error

// Remove rule by ID
func (m *Manager) RemoveRule(id string) bool

// Clear all rules
func (m *Manager) Clear()

// Count total rules
func (m *Manager) Count() int

// Export all rules to single RuleSet
func (m *Manager) Export() *RuleSet

// Save rules to file
func (m *Manager) SaveWorkspaceRules(path string) error
func (m *Manager) SaveProjectRules(path string) error
```

Rule Methods

```
// Check if rule matches file path
func (r *Rule) Matches(filePath string) bool

// Calculate match score
func (r *Rule) MatchScore(filePath string) int

// Validate rule
func (r *Rule) Validate() error

// Clone rule (deep copy)
func (r *Rule) Clone() *Rule

// String representation
func (r *Rule) String() string
```

Parser Functions

```
// Create new parser for file
func NewParser(filePath string) *Parser

// Parse .clinerules file
func (p *Parser) Parse() (*RuleSet, error)

// Parse rules from string
func ParseString(content string) (*RuleSet, error)

// Format RuleSet back to .clinerules format
func Format(ruleSet *RuleSet) string
```

Troubleshooting

Common Issues

Issue 1: Rules Not Matching Files

Symptoms: GetRulesForFile() returns empty array

Causes: - Pattern doesn't match file path - PatternType not set (defaults to empty string) - Rule validation failed

Solutions:

```
// 1. Check pattern matches
rule := &rules.Rule{
    Pattern:      "src/**/*.go",
    PatternType:  rules.PatternTypeGlob, // REQUIRED!
}
matches := rule.Matches("src/api/handler.go")
fmt.Printf("Matches: %v\n", matches)

// 2. Validate rule
if err := rule.Validate(); err != nil {
    fmt.Printf("Validation error: %v\n", err)
}

// 3. Check rule loading
manager := rules.NewManager()
if err := manager.LoadFromDirectory("."); err != nil {
    fmt.Printf("Load error: %v\n", err)
}
fmt.Printf("Loaded %d rules\n", manager.Count())
```

Issue 2: Wrong Rule Priority

Symptoms: Lower priority rule appears first

Causes: - Inheritance scoring overrides explicit priority - Pattern specificity affects score

Solutions:

```
// Check match scores
matches := manager.GetRulesForFile("main.go")
for _, match := range matches {
    fmt.Printf("%s: score=%d, priority=%d\n",
        match.Rule.Name, match.Score, match.Rule.Priority)
}

// Use higher priority for important rules
rule := &rules.Rule{
    Name:      "Critical Rule",
    Priority: 100, // Very high priority
}
```

Issue 3: Pattern Not Matching Expected Files

Symptoms: Glob pattern doesn't match files

Causes: - Misunderstanding of glob wildcards - Directory separators on Windows

Solutions:

```
// Test pattern matching
testFiles := []string{
    "src/main.go",
    "src/api/handler.go",
    "test/main.go",
}

for _, file := range testFiles {
    matches := rule.Matches(file)
    fmt.Printf("%s: %v\n", file, matches)
}

// Use ** for directory depth
pattern: src/**/*.go    // Matches all .go files under src/
pattern: *.go            // Matches .go files in current directory
                        only
```

Issue 4: Rule Parse Errors

Symptoms: Parse() returns error

Causes: - Invalid rule format - Missing required fields - Invalid regex pattern

Solutions:

```
parser := rules.NewParser(".clinerules")
ruleSet, err := parser.Parse()
if err != nil {
    fmt.Printf("Parse error: %v\n", err)
    // Check file syntax
}

// Validate each rule
for _, rule := range ruleSet.Rules {
    if err := rule.Validate(); err != nil {
        fmt.Printf("Rule '%s' invalid: %v\n", rule.Name, err)
    }
}
```

Debug Mode

Enable verbose logging:

```
// Log all rule matches
matches := manager.GetRulesForFile(filePath)
fmt.Printf("Found %d matching rules for %s:\n", len(matches),
    filePath)
for i, match := range matches {
    fmt.Printf("  %d. %s (score: %d, priority: %d)\n",
        i+1, match.Rule.Name, match.Score, match.Rule.Priority)
}

// Log all loaded rules
allRules := manager.GetAllRules()
fmt.Printf("Total rules loaded: %d\n", len(allRules))
for _, rule := range allRules {
    fmt.Printf("  - %s: pattern=%s, type=%s\n",
        rule.Name, rule.Pattern, rule.PatternType)
}
```

FAQ

General Questions

Q: What's the difference between Cline Rules and linters?

A: Linters check code syntax and style after it's written. Cline Rules guide LLMs *while generating code*, ensuring it follows project conventions from the start. They work together: rules guide generation, linters verify compliance.

Q: Can I use Cline Rules without LLMs?

A: Yes! Rules can be used for: - Documentation (centralized project guidelines) - Code review checklists - Onboarding new developers - Style guide reference

Q: How many rules should I have?

A: Start with 5-10 essential rules, then grow organically. Too many rules can overwhelm developers and LLMs. Focus on: - Critical security requirements - Common mistake prevention - Project-specific patterns

Q: Do rules slow down code generation?

A: Minimal impact. Rule loading is fast (<100ms for typical projects), and matching is optimized. The quality improvement far outweighs any performance cost.

Pattern Matching

Q: What's the difference between * and **?

A: - * matches files in current directory: *.go → main.go ✓, api/main.go ✗ - ** matches at any depth: **/*.go → main.go ✓, api/main.go ✓

Q: How do I match files in subdirectories?

A: Use **/:

```
pattern: src/**/*.go      # All .go files under src/
pattern: **/test/*.go     # All .go files in any test/ directory
pattern: **/*_test.go     # All test files anywhere
```

Q: Can I combine multiple patterns?

A: No, but you can create multiple rules:

```
[Go Files]
pattern: *.go
Content for Go files
```

```
[Go Test Files]
pattern: *_test.go
Content for test files (inherits from Go Files)
```

Q: Do patterns support OR logic?

A: Use regex patterns:

```
pattern: /\.*\.(go|js|ts)$/  # Matches .go, .js, or .ts files
```

Inheritance

Q: Can file-specific rules completely replace project rules?

A: No, all matching rules are applied. File rules are *added* with higher priority, not replacing. To override, use the same rule name with higher priority.

Q: What if I don't want inheritance?

A: Use very specific patterns in lower levels:

```
# Project rule
[General Go]
pattern: *.go
priority: 1
```

```
# File rule (overrides by higher priority)
[Special Go]
pattern: api/*.go
priority: 100 # Much higher, effectively replaces
```

Q: How do I share rules across multiple projects?

A: Use workspace-level rules in `.helix/.clinerules`:

```
myworkspace/
├── .helix/
│   └── .clinerules          # Shared across all projects
├── project1/
│   └── .clinerules          # Project-specific
└── project2/
    └── .clinerules          # Project-specific
```

Best Practices

Q: Should I commit `.clinerules` to git?

A: **Yes!** Rules are part of your project's definition, like linter configs or style guides. Team members and CI should use the same rules.

Q: How often should I update rules?

A: Update rules when: - New patterns or requirements emerge - Common mistakes are discovered - Architecture or conventions change - Post-incident analysis reveals gaps

Q: Can rules be too strict?

A: Yes. Signs of over-strict rules: - Developers constantly asking for exceptions - Rules conflict with each other - LLMs produce overly verbose code - Development slows significantly

Balance is key. Start permissive, tighten as needed.

Q: Should every file have rules?

A: No. Focus rules on: - Critical code (security, payments, data handling) - Complex patterns (architecture, integrations) - Common mistakes (error handling, validation)

Generic files (utilities, helpers) often don't need specific rules.

Conclusion

The Cline Rules System provides a powerful, flexible way to ensure consistent, high-quality code generation across your project. By using the three-level hierarchy, pattern matching, and priority system, you can create precise guidelines that LLMs automatically follow.

Key Takeaways

1. **Start Simple:** Begin with a few essential rules, expand as needed
2. **Use Hierarchy:** Workspace → Project → File for proper organization
3. **Be Specific:** Clear, actionable rules produce better results
4. **Test Patterns:** Verify patterns match expected files
5. **Iterate:** Update rules based on real-world usage

Next Steps

1. Create your first `.clinerules` file
2. Define 3-5 essential rules for your project
3. Test with LLM code generation
4. Refine based on results
5. Expand coverage to critical areas

Resources

- **GitHub:** [HelixCode Repository](#)
- **Documentation:** [Full API Docs](#)
- **Examples:** [Example Rules](#)
- **Community:** [Discord Server](#)

Document Version: 1.0 **Last Updated:** November 7, 2025 **Next Feature:** Focus Chain System (Phase 2, Feature 3)

End of Cline Rules System User Guide